

REGISTRO DE MOBILE - SEMANA 21

Roteiro 1 — Iterando com base em feedback

1. Quais métodos você utilizaria para coletar feedback dos usuários?

Eu utilizaria formulários online, como Google Forms ou Typeform, pesquisas de satisfação, avaliações dentro do sistema e dados de uso da aplicação. Também poderia utilizar chatbots ou formulários rápidos para receber opiniões em tempo real.

2. Como você categorizaria os problemas encontrados?

Os problemas poderiam ser classificados de acordo com sua gravidade e impacto:

- **Alta prioridade:** problemas que impedem ou dificultam muito a realização de uma tarefa.
- **Média prioridade:** problemas que prejudicam a experiência, mas não impedem o uso.
- **Baixa prioridade:** pequenos problemas ou sugestões de melhoria.

3. Qual a importância de iterar com base no feedback?

A iteração permite melhorar o sistema continuamente de acordo com as necessidades reais dos usuários. Dessa forma, é possível corrigir problemas, melhorar a experiência e evitar que erros continuem presentes no produto.

4. Como você validaria se as mudanças resolveram o problema?

Eu compararia os resultados antes e depois das mudanças, analisando os dados de uso e realizando novos testes com os usuários. Também verificaria se a taxa de abandono do processo diminuiu e se os usuários conseguiram concluir o checkout com mais facilidade.

Resposta ao cenário — Abandono no checkout

Para resolver o problema de abandono no checkout, eu seguiria estas etapas:

1. **Coletar feedback** dos usuários por meio de formulários, pesquisas e análise dos dados do sistema.
2. **Identificar as principais dificuldades** encontradas durante o checkout.
3. **Categorizar os problemas** de acordo com sua prioridade e impacto.
4. **Propor melhorias**, como simplificar o processo de compra, deixar as informações mais claras e reduzir etapas desnecessárias.
5. **Implementar as mudanças** durante os ciclos de desenvolvimento.
6. **Realizar novos testes** com usuários para verificar se o problema foi solucionado.
7. **Analisar os resultados** e continuar fazendo ajustes quando necessário.

Dessa forma, o sistema pode ser melhorado continuamente com base nas necessidades e no comportamento dos usuários.

Roteiro 2 — Identificando e refatorando códigos ineficientes

1. Como você identificaria áreas do código que precisam de refatoração?

Eu analisaria o código procurando trechos repetidos, funções muito grandes ou complexas, nomes pouco claros e falta de organização. Também poderia utilizar ferramentas como o SonarQube para identificar problemas de qualidade e possíveis melhorias.

2. Quais técnicas você utilizaria para melhorar a legibilidade e a manutenção do código?

Eu utilizaria nomes mais claros para variáveis e funções, dividiria funções grandes em partes menores, eliminaria códigos repetidos e simplificaria estruturas muito complexas. Também manteria um padrão de organização no projeto.

3. Por que é importante refatorar sem alterar o comportamento funcional?

Porque a refatoração deve melhorar a estrutura interna do código sem modificar o que o sistema faz. Assim, evitamos criar novos erros ou prejudicar funcionalidades que já estão funcionando corretamente.

4. Como você documentaria as mudanças realizadas?

Eu registraria as principais alterações feitas, explicando quais partes foram modificadas e qual era o objetivo de cada mudança. Também utilizaria comentários quando fossem realmente necessários e registraria as alterações no Git para facilitar o acompanhamento do projeto.

Resposta ao cenário — Plano de ação

Para melhorar um código difícil de entender e que está atrasando o desenvolvimento, eu seguiria estas etapas:

1. **Analisar o código** e identificar os trechos mais complexos ou repetidos.
2. **Utilizar ferramentas como o SonarQube** para encontrar problemas de qualidade.
3. **Definir prioridades**, começando pelos problemas que mais prejudicam o desenvolvimento.
4. **Refatorar o código**, utilizando funções menores, nomes mais claros e evitando duplicações.
5. **Realizar testes** para garantir que as funcionalidades continuam funcionando corretamente.
6. **Documentar as alterações** realizadas.
7. **Registrar as mudanças no Git** para manter o histórico do projeto.

Dessa forma, o código fica mais organizado, fácil de entender e manter, sem alterar suas funcionalidades.

Roteiro 3 — Planejando e documentando projetos de interface

1. Quais etapas devem ser consideradas no planejamento de um projeto de interface?

Primeiro, é necessário entender as necessidades dos usuários e definir os objetivos do projeto. Depois, devem ser organizados os conteúdos e fluxos de navegação e criados wireframes para representar a estrutura das telas.

2. Como garantir que o projeto atenda às necessidades do cliente?

É importante conhecer as expectativas do cliente e dos usuários, criar uma interface simples e intuitiva e realizar testes durante o desenvolvimento. O feedback recebido deve ser utilizado para fazer melhorias no projeto.

3. Por que é importante documentar o projeto?

A documentação registra as decisões tomadas, as funcionalidades e as justificativas do projeto. Isso facilita a comunicação entre a equipe e o cliente e também ajuda em futuras alterações e manutenções.

4. Como apresentar o projeto aos stakeholders?

Eu apresentaria o projeto mostrando os objetivos, o público-alvo, os wireframes, o fluxo de navegação e as principais funcionalidades. Também explicaria as decisões de design e apresentaria os resultados dos testes e melhorias realizadas.

Resposta ao cenário — Interface para acesso a relatórios financeiros

Para desenvolver uma interface que facilite o acesso aos relatórios financeiros, eu seguiria estas etapas:

1. **Planejamento:** identificar as necessidades dos usuários e definir os objetivos da interface.
2. **Organização:** definir as informações que serão apresentadas e criar o fluxo de navegação.
3. **Prototipagem:** criar wireframes e protótipos das telas, utilizando ferramentas como o Figma.
4. **Desenvolvimento:** implementar a interface de acordo com o planejamento.
5. **Testes:** testar a interface com usuários e identificar possíveis dificuldades.
6. **Melhorias:** utilizar os feedbacks recebidos para corrigir problemas e melhorar a experiência.
7. **Documentação:** registrar as decisões, funcionalidades e alterações realizadas.
8. **Apresentação:** apresentar o resultado ao cliente e aos stakeholders, explicando as escolhas feitas e os resultados dos testes.

Dessa forma, o projeto fica organizado, documentado e alinhado às necessidades dos usuários e do cliente.